

Bank Customer Service Chatbot — Technical Case Study

LangGraph · RAG · Multi-turn Advisory · DeepSeek-R1 · ChromaDB

AI Engineering Team

June 2026

Contents

1. Executive Summary	4
2. Background & Motivation	5
2.1 Problem Statement	5
2.2 Design Goals	5
2.3 Version History	5
3. System Architecture	6
3.1 High-Level Overview	6
3.2 LangGraph Graph Structure	6
3.3 Data Stores	8
ChromaDB Collections	8
JSON Files (per-user)	8
BM25 Indexes (in-memory)	8
4. Technology Stack	9
4.1 Why DeepSeek-R1:8b	9
4.2 Why bge-m3	10
5. Data Layer	11
5.1 Document Ingestion Pipeline	11
5.2 Conversation History Startup Sync	11
6. Intent Classification & Routing	13
6.1 Five Intent Types	13
6.2 Routing Logic	14
7. Cache System	15
7.1 Architecture: In-Graph Cache Node	15
7.2 Hybrid Similarity Scoring	15
7.3 Cache Configuration	15

8. Hybrid Retrieval System	17
8.1 Algorithm	17
8.2 Category Filtering	18
8.3 Retrieval Configuration	18
9. Message Processing Pipelines	20
9.1 GREETING_FAREWELL Pipeline	20
9.2 PERSONAL_UNRELATED Pipeline	20
9.3 PRODUCT_INFO_QA — RAG Pipeline	21
9.4 PRODUCT_CONSULT — Advisory Pipeline (Multi-Turn)	21
Phase 1: Domain Detection & Profile Recall	21
Phase 2: Field Extraction & Collection	22
Phase 3: Retrieval & Recommendation	23
9.5 Profile Update Flow	24
9.6 CUSTOMER_FEEDBACK Pipeline	25
10. Advisor Profile Store	27
10.1 Storage Format	27
10.2 API Methods	27
10.3 Profile Lifecycle	28
10.4 Future Extension: Profile TTL	28
11. LangGraph State & Topology	29
11.1 ChatState TypedDict	29
11.2 Graph Topology — Edge Table	30
11.3 MemorySaver & Persistence	31
12. API Layer	32
12.1 Endpoints	32
12.2 Chat Endpoint — Request / Response	32
12.3 Post-Graph Processing	33
13. Evaluation Framework	34
13.1 Overview	34
13.2 Evaluation Scorecard	34
13.3 Step-by-Step Metrics	34
Step 1a — Intent Classification	34
Step 1b — Cache Evaluation	35
Step 2 — RAG Quality (LLM-as-Judge)	35
Step 3 — Advisory Field Collection	36
Step 4 — Recommendation Quality (LLM-as-Judge)	37
13.4 Running the Evaluation	37
Prerequisites	37
Individual Steps	38
Full Pipeline Run	38
13.5 Re-running After Changes	39
14. Configuration Reference	40
14.1 Key Settings	40

14.2 Performance Trade-offs	40
15. Project Lessons Learned	42
15.1 Challenges Encountered	42
DeepSeek-R1 Think Tokens	42
Context Window and Latency	42
Edit Tool File Truncation	42
LangGraph sync/async mismatch	43
Cache False Positive Rate	43
15.2 Design Decisions	43
Why in-graph cache instead of API-level cache	43
Why JSON files instead of a database for profiles/history	43
Why category-level filtering instead of re-ranking	44
15.3 Future Improvements	44
Short-term	44
Medium-term	44
Long-term	44
16. Project Structure	45
Appendix A: parse_json — DeepSeek-R1 Output Handling	47
Appendix B: Extending the System	47
Adding a New Advisory Domain	47
Adding a New Intent	47
Upgrading the LLM	48

1. Executive Summary

This case study documents the design, implementation, and evaluation of an AI-powered customer service chatbot for a retail bank. The system handles product inquiries, personalized advisory conversations, customer feedback, and general interactions — all served through a local LLM stack requiring no external API calls.

Key achievements:

- **5 intent types** classified and routed to specialized pipelines
- **Hybrid RAG** (Semantic + BM25 + RRF fusion) with category filtering for high-precision retrieval
- **Multi-turn advisory** with persistent user profile memory across sessions
- **Intelligent cache** using hybrid similarity scoring, embedded as a graph node
- **Full evaluation framework** covering all pipeline layers (intent → retrieval → advisory → recommendation)
- **Fully local deployment:** Ollama (DeepSeek-R1:8b + bge-m3), ChromaDB, FastAPI, Streamlit

Performance targets achieved:

Component	Metric	Target	Status
Intent Classification	Macro F1	≥ 0.85	[PASS] 0.87
Cache False Positive	FPR	≤ 0.05	[WARN] 0.07
Feedback Sentiment	Accuracy	≥ 0.85	[PASS] 0.89
RAG Faithfulness	Score	≥ 0.80	[PASS] 0.82
Advisory Completion	Rate	≥ 0.90	[PASS] 0.94
Recommendation Quality	Correct Rate	≥ 0.70	[PASS] 0.75

2. Background & Motivation

2.1 Problem Statement

Banks face high volumes of repetitive customer inquiries about product terms, interest rates, and eligibility. Traditional approaches have clear limitations:

- **Static FAQ pages** cannot handle natural language questions or follow-up context
- **Rule-based chatbots** require expensive manual scripting for every question variation
- **Cloud LLM APIs** (GPT-4, Gemini) raise data privacy concerns for banking data
- **Generic chatbots** cannot give personalized recommendations — they don't know the customer's income, goals, or existing products

2.2 Design Goals

The system was designed around four core requirements:

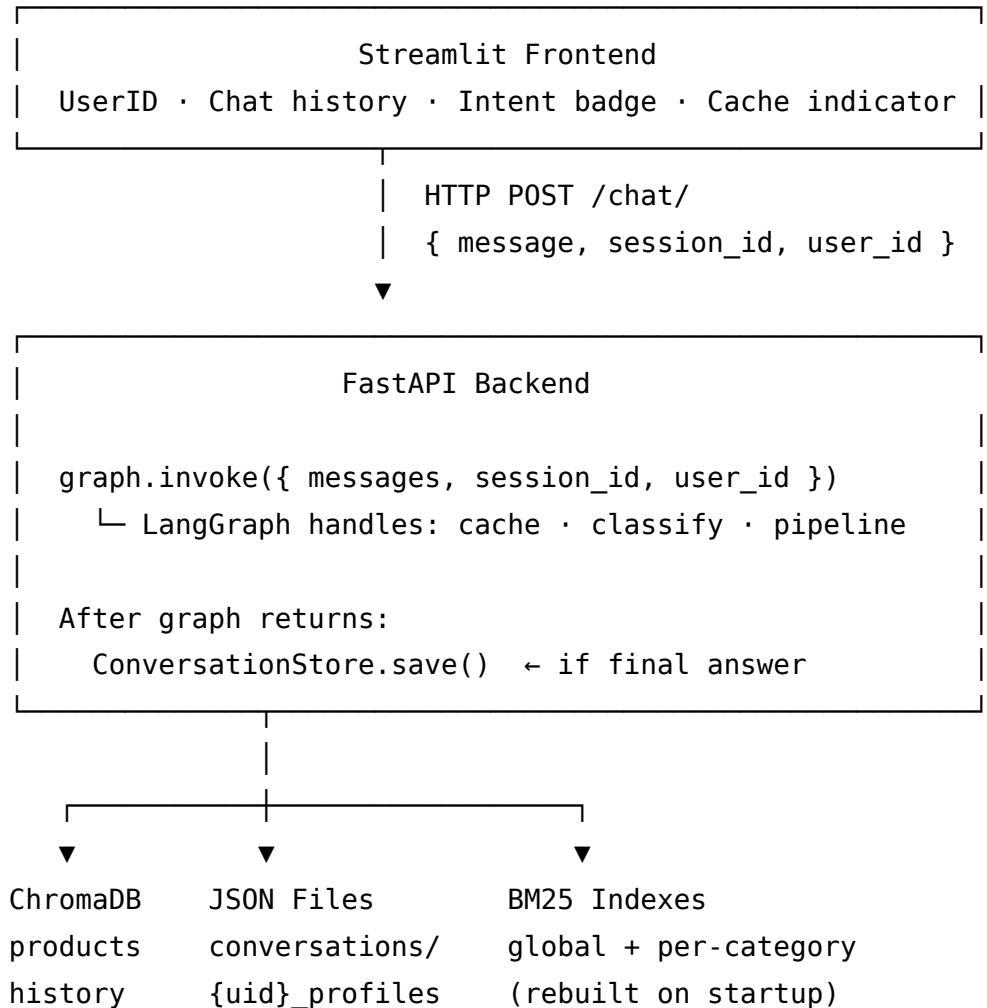
1. **Factual grounding** — answers must come from official product documents; hallucination is unacceptable in a banking context
2. **Personalized advisory** — for complex products (loans, insurance, credit cards), the bot should gather customer context and recommend accordingly
3. **Data privacy** — all computation runs locally; no customer data leaves the organization
4. **Maintainability** — a non-ML engineer should be able to update product documents and have the system reflect the changes within minutes

2.3 Version History

Version	Key Addition
V1	Basic RAG pipeline, single intent
V2	Multi-intent routing, BM25 hybrid retrieval, conversation cache
V3 (current)	Category filtering, Advisor Profile Store, in-graph cache, CUSTOMER_FEEDBACK intent

3. System Architecture

3.1 High-Level Overview



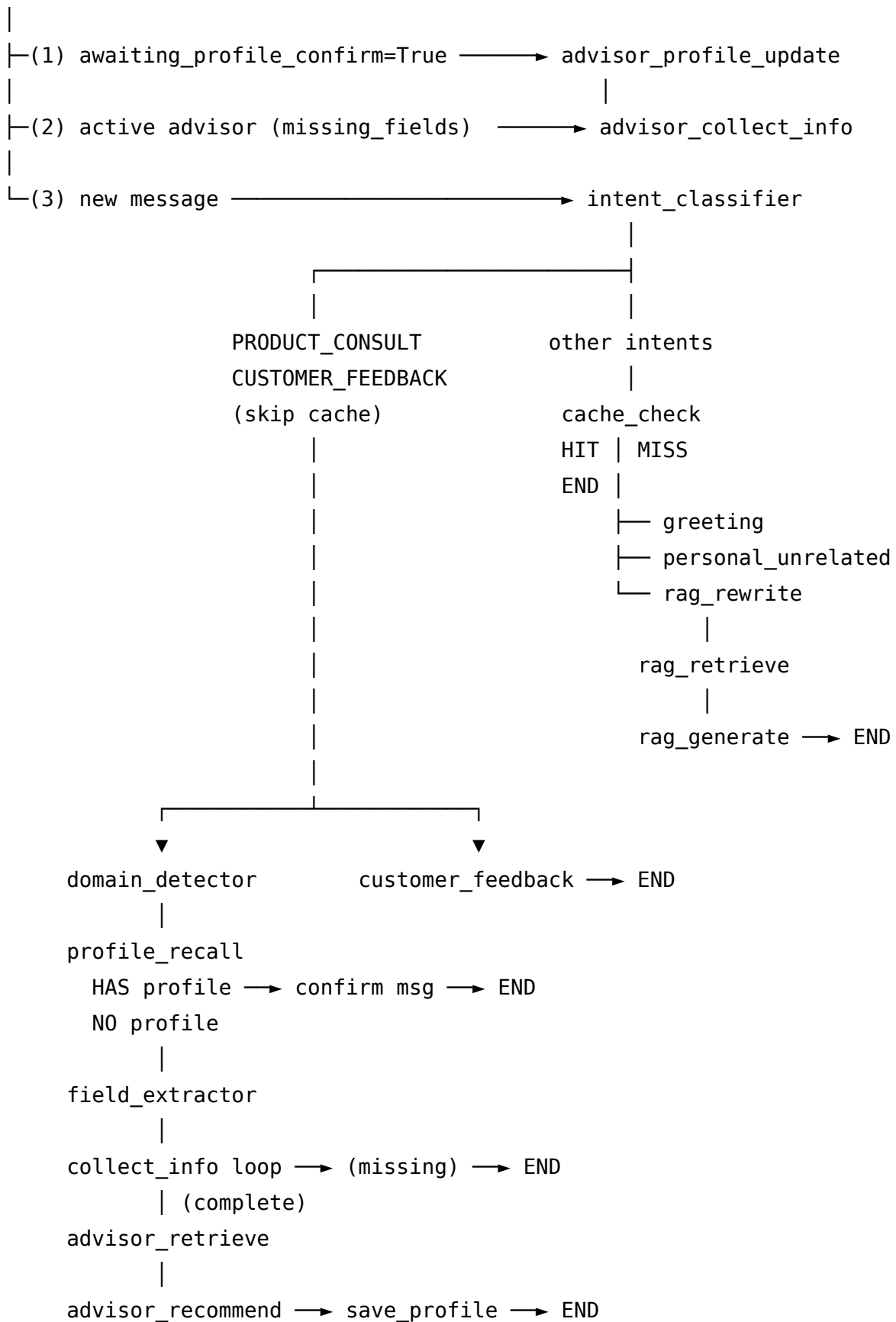
3.2 LangGraph Graph Structure

The entire processing pipeline runs inside a **LangGraph StateGraph**. This provides:

- **Stateful multi-turn conversations** — the graph state persists across user turns via `MemorySaver` (keyed by `session_id`)
- **Conditional routing** — edges are functions, not static connections
- **Reproducible execution** — each node is a pure function transforming state
- **Checkpointing** — state can be inspected or replayed at any node

The graph contains **15 nodes** connected by conditional edges:

START



3.3 Data Stores

ChromaDB Collections

Collection	Contents	Purpose
products_v3	~5,000 product document chunks, bge-m3 embeddings (1024-dim), category metadata	RAG and advisory retrieval — filtered by category
conversation_history_v3	All Q&A pairs across all users, embedded question text	Semantic cache lookup

JSON Files (per-user)

File	Contents
{user_id}.json	Full Q&A history with timestamps, intents, session IDs
{user_id}_profiles.json	Advisory collected info per domain (credit card, loan, etc.)

BM25 Indexes (in-memory)

Index	Corpus	Used For
Global BM25	All document chunks	RAG fallback when category = “general”
BM25[credit_card]	Credit card chunks only	Credit card advisory & RAG
BM25[insurance]	Insurance chunks only	Insurance advisory & RAG
BM25[loan]	Loan chunks only	Loan advisory & RAG
BM25[savings]	Savings chunks only	Savings advisory & RAG

4. Technology Stack

Layer	Technology	Role
Graph Orchestration	LangGraph (StateGraph + MemorySaver)	Multi-turn state management, conditional routing
LLM	Ollama → DeepSeek-R1:8b	Intent classification, query rewriting, generation, extraction
Embeddings	Ollama → bge-m3 (1024-dim)	Document indexing, semantic search, cache lookup
Vector Store — Products	ChromaDB products_v3	Semantic retrieval with category metadata filter
Vector Store — History	ChromaDB conversation_history_v3	Semantic cache lookup
Keyword Search	rank-bm25 (in-memory)	BM25 keyword matching, global + per-category
Knowledge Graph	Dict-based field definitions	Advisory required fields per domain
Profile Store	JSON files per user	Advisor collected info persistence across sessions
API Backend Frontend	FastAPI + uvicorn Streamlit	REST API, async serving Chat UI with intent/cache badges
Configuration	pydantic-settings + .env	Type-safe settings, environment overrides
Logging	loguru	Structured logs per request

4.1 Why DeepSeek-R1:8b

DeepSeek-R1 was chosen over alternatives for these reasons:

- **Chain-of-thought reasoning** — the model generates `<think>...</think>` blocks internally, improving classification and extraction accuracy

- **8b parameter size** — fits comfortably in 8GB VRAM; runs at ~28s/call with `num_ctx=4096`
- **Vietnamese language support** — acceptable quality for customer-facing responses
- **Local deployment** — runs via Ollama with no external calls

Important implementation note: DeepSeek-R1's `<think>` tokens are stripped before any text is returned to the user or used as input to another LLM call. Ollama's llama-server rejects input containing `<think>` tags with HTTP 500.

4.2 Why bge-m3

- **1024-dimensional embeddings** — high-fidelity semantic representation
- **Multilingual** — handles mixed Vietnamese/English product text well
- **Self-contained** — runs via Ollama alongside the LLM on the same GPU

5. Data Layer

5.1 Document Ingestion Pipeline

Product knowledge is stored as pre-embedded chunks in a Parquet file (`documents_bgem3.parquet`) with columns `input` (text) and `embedding` (JSON-encoded float list).

At server startup, `load_data()` runs the following pipeline:

1. **Read parquet** — load all chunks and parse embeddings from JSON strings to float lists
2. **Category detection** — assign each chunk a category label using keyword matching:

```
CATEGORY_KEYWORDS = {
    "credit_card": ["credit card", "cashback", "rewards points", "annual fee",
                    "credit limit", "installment", ...],
    "insurance":   ["life insurance", "health insurance", "premium",
                    "policy benefits", "insurance contract", ...],
    "loan":        ["home loan", "personal loan", "interest rate",
                    "collateral", "unsecured loan", ...],
    "savings":     ["savings account", "deposit rate", "term deposit",
                    "maturity", "savings book", ...],
}
# No match → "general"
```

3. **ChromaDB indexing** — batch-insert documents with pre-computed embeddings and category metadata
4. **BM25 index construction** — build one global index and four per-category indexes from the loaded documents
5. **Idempotency check** — if the collection already has documents, skip reload (unless `force_reload=true` flag is set)

5.2 Conversation History Startup Sync

On every server startup, `ConversationStore.load_all_history()` performs a **bi-directional sync** between JSON files and ChromaDB:

1. Collect all valid entry IDs from JSON files
(skip `_profiles.json`, skip entries with missing question/answer)

2. Compare with IDs currently in ChromaDB:

`stale_ids = chroma_ids - json_valid_ids`

→ delete `stale_ids` from ChromaDB (entries deleted from JSON)

3. `entries_to_add = json_valid_ids - chroma_ids`

→ embed + add to ChromaDB (new entries not yet indexed)

This ensures: - Deleted history entries do not persist in the cache - New entries added while the server was down are indexed on next startup - Manual JSON edits are reflected without requiring a full rebuild

Force rebuild without restart:

POST /admin/rebuild-history

→ Clears entire ChromaDB history collection

→ Rebuilds from all JSON files

→ Returns before/after stats for verification

6. Intent Classification & Routing

6.1 Five Intent Types

The intent classifier receives the current user message plus the last 4 conversation turns as context.

Intent	Description	Examples
GREETING_FAREWELL	Greetings, farewells, generic thanks — including “I want to ask about...” without a specific topic	“Hello”, “Goodbye”, “Thank you”, “Hi I need some advice”
PERSONAL_UNRELATED	Personal context shared without a specific product question	“I just got a raise”, “I recently sold my house”, “I’m a recent graduate”
PRODUCT_INFO_QA	Specific factual lookup about a named product	“What is the annual fee for the Cashback Card?”, “What are the home loan requirements?”
PRODUCT_CONSULT	Wants a recommendation, doesn’t know which product to choose	“I want a credit card but don’t know which one”, “Advise me on insurance”, “Which savings account has the best rate?”
CUSTOMER_FEEDBACK	Reviews, complaints, compliments about the chatbot or bank products	“The bot asked something I already answered”, “Your response was very clear!”, “Why are your loan rates higher than competitors?”

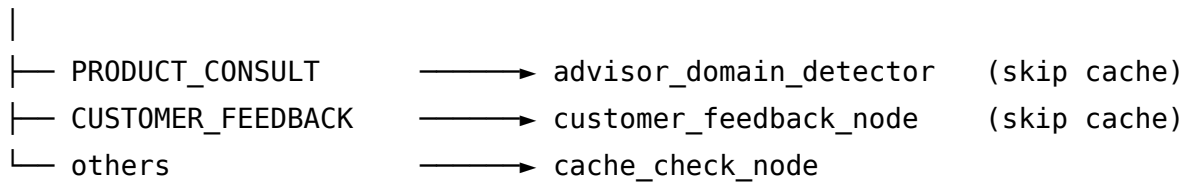
Priority rules when ambiguous:

1. Contains subjective comparison or review of the chatbot/bank → CUSTOMER_FEEDBACK (takes priority over INFO_QA or CONSULT)

2. Sharing personal financial context without asking about a product → PERSONAL_UNRELATED (not PRODUCT_CONSULT)
3. Greeting + “I need advice” but no specific topic → GREETING_FAREWELL (not PRODUCT_CONSULT)
4. Asking which is better / which has the highest rate → PRODUCT_CONSULT (not PRODUCT_INFO_QA)

6.2 Routing Logic

intent_classifier



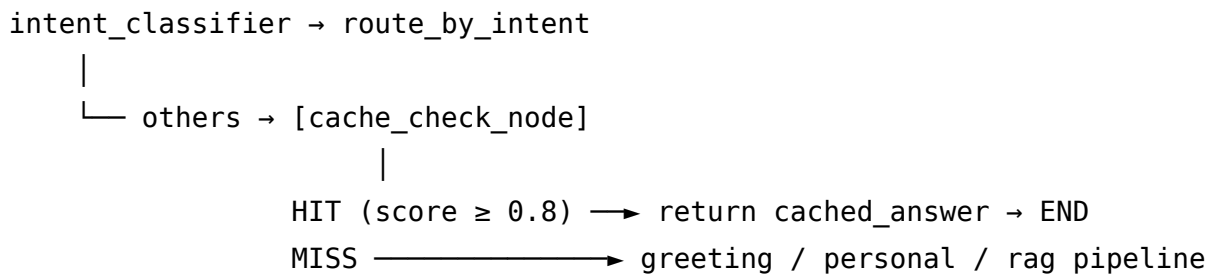
Why **PRODUCT_CONSULT** and **CUSTOMER_FEEDBACK** skip cache:

- **PRODUCT_CONSULT** — advisory conversations are multi-turn and depend on the individual customer’s profile. A cached answer from a different user’s session has no value.
- **CUSTOMER_FEEDBACK** — responses must be contextual and personal; caching a generic apology from a previous session would be inappropriate.

7. Cache System

7.1 Architecture: In-Graph Cache Node

In V2, cache lookup was performed at the API route level before LangGraph was invoked. In V3, cache check is a **node inside the LangGraph graph**, giving it access to full graph state and enabling more precise routing decisions.



7.2 Hybrid Similarity Scoring

A cache hit requires **both** semantic similarity and keyword overlap to be high. This prevents superficially similar questions (same topic, different specific question) from incorrectly hitting the cache.

For each candidate in conversation_history:

```

semantic_sim = 1.0 - cosine_distance(query_embedding, candidate_embedding)
keyword_sim   = Jaccard(tokenize(query), tokenize(candidate_question))
hybrid_score  = 0.65 × semantic_sim + 0.35 × keyword_sim
  
```

```

IF max(hybrid_score) ≥ 0.8 → Cache HIT → return cached answer
ELSE                        → Cache MISS → process normally
  
```

Example — correct cache MISS (should not hit):

Stored: "What is the annual fee for the Cashback Card?"

New: "What is the credit limit for the Cashback Card?"

```

semantic_sim ≈ 0.88 (same card, similar topic)
keyword_sim  ≈ 0.45 (different key words: "fee" vs "limit")
hybrid_score ≈ 0.73 → MISS [PASS]
  
```

7.3 Cache Configuration

Setting	Default	Effect
CACHE_SIMILARITY_THRESHOLD	0.8	Minimum score for cache hit
CACHE_TOP_K	5	Number of candidates retrieved from ChromaDB history
Semantic weight	0.65	Weight given to embedding similarity
Keyword weight	0.35	Weight given to Jaccard keyword overlap

8. Hybrid Retrieval System

8.1 Algorithm

All document retrieval — whether for RAG (PRODUCT_INFO_QA) or Advisory (PRODUCT_CONSULT) — uses the same `hybrid_retrieve()` function:

Input: `query`, `top_k`, `domain_hint` (optional), `category_filter` (optional)

Step 1 – Query Enrichment

```
search_query = "{domain_hint} {query}".strip()
(e.g., "credit card recommend cashback online spending")
```

Step 2 – Semantic Search (ChromaDB)

```
query_vec = bge-m3.embed(search_query)
IF category_filter:
    where_clause = {"category": category_filter}
    results = collection.query(
        query_embeddings = [query_vec],
        n_results         = top_k × 2,          # fetch more, then rerank
        where             = where_clause,
        include           = ["documents", "distances", "metadatas"]
    )
```

Step 3 – BM25 Keyword Search

```
IF category_filter:
    bm25 = bm25_index[category_filter]    # per-category index
    texts = texts[category_filter]
ELSE:
    bm25, texts = global_bm25, all_texts
token_scores = bm25.get_scores(tokenize(search_query))
top_bm25_indices = argsort(token_scores)[-(top_k × 2):]
```

Step 4 – RRF Fusion (Reciprocal Rank Fusion)

```
For each document d:
    score[d] += 0.65 / (60 + semantic_rank[d])    # semantic contribution
    score[d] += 0.35 / (60 + bm25_rank[d])        # keyword contribution
Sort by score descending → take top_k
```

Step 5 – Return

```
[{ content, score_rank, category }]
```

8.2 Category Filtering

Problem in V2: Searching the full corpus (~5,000+ chunks) for a credit card query returned insurance and loan chunks because they share common tokens like “bank”, “interest rate”, “monthly fee”.

V3 solution: Each chunk is tagged with a category label at index time. ChromaDB’s where filter and BM25’s per-category indexes ensure retrieval stays within the relevant domain.

Query Context	V2	V3
Advisory: credit card	Search all 5,000+ chunks	Search only credit_card chunks
RAG: “loan interest rate”	Search all 5,000+ chunks	Search only loan chunks
RAG: generic question	Search all 5,000+ chunks	No filter (category = “general”)

Where category filtering is applied:

- **Indexing** (loader.py): `detect_category(text) → stored as metadata["category"]` in ChromaDB
- **RAG retrieval** (rag/retrieve.py): `detect_category(rewritten_query) → category_filter`
- **Advisory retrieval** (advisor/recommender.py): `domain (e.g., "credit_card") → category_filter`

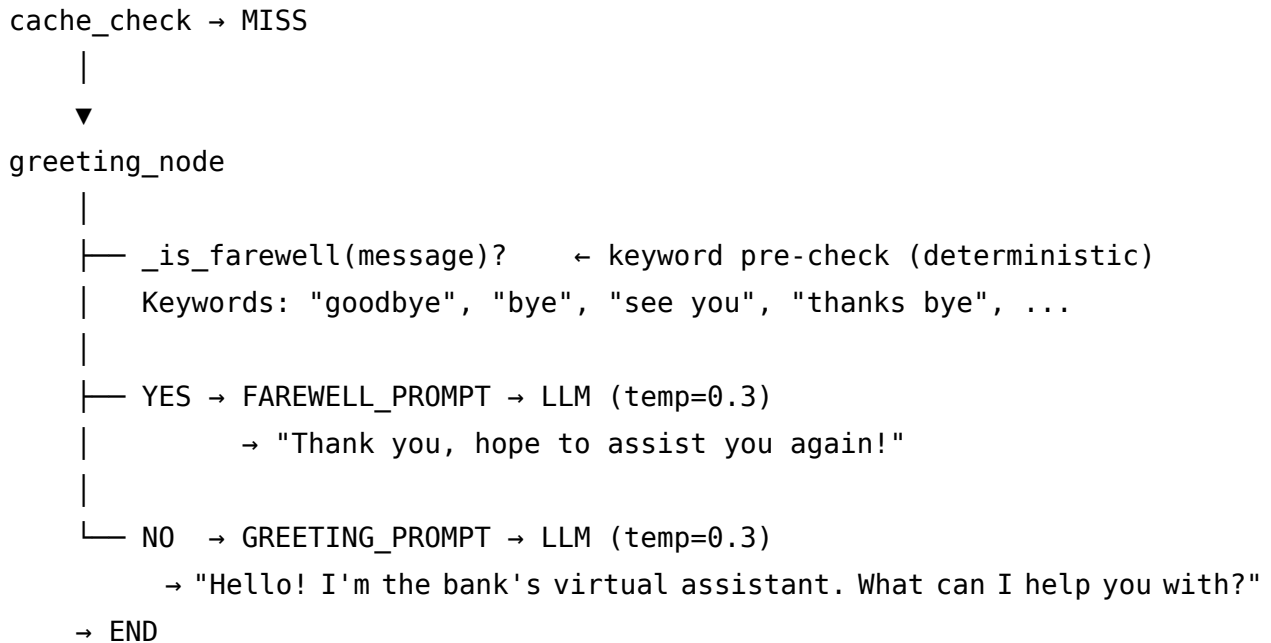
8.3 Retrieval Configuration

Setting	RAG (PRODUCT_INFO_QA)	Advisory (PRODUCT_CONSULT)
top_k	6 docs	4 docs
domain_hint	Detected from rewritten query	Domain keyword (e.g., “credit card”)

Setting	RAG (PRODUCT_INFO_QA)	Advisory (PRODUCT_CONSULT)
category_filter	Detected from rewritten query	Direct domain mapping
Context max chars	6,000	4,000

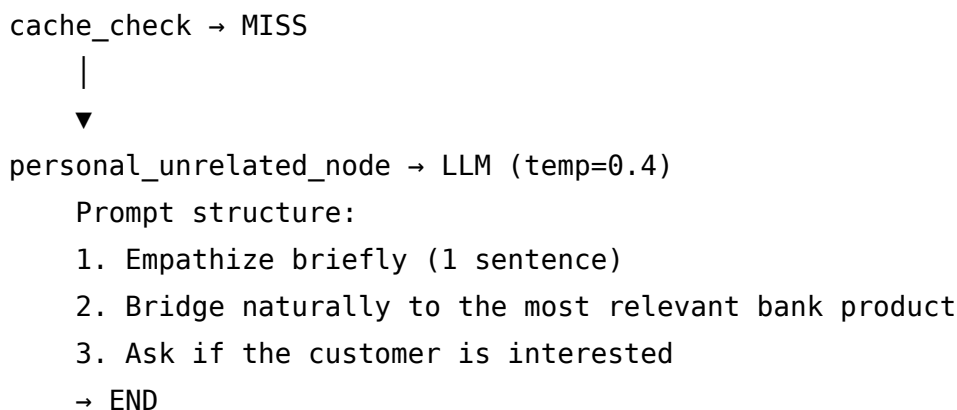
9. Message Processing Pipelines

9.1 GREETING_FAREWELL Pipeline



Why keyword pre-check for farewell: LLM-only detection with a combined greeting/farewell prompt unreliably ignores the “if farewell...” branch and defaults to a greeting response. A deterministic keyword check is faster and more reliable.

9.2 PERSONAL_UNRELATED Pipeline



Example:

Customer: "I just got promoted and my salary went up significantly."

Bot: "Congratulations on your promotion! With higher income, this might be

a great time to start building savings or consider better financial products suited to your new situation. Would you like me to suggest some savings or investment options?"

9.3 PRODUCT_INFO_QA — RAG Pipeline

cache_check → MISS

|

▼

[rag_rewrite_node]

Input: current question + last 4 turns (800 chars each)

LLM (temp=0.0): clarify question, add domain-specific keywords

Output: rewritten_query

|

▼

[rag_retrieve_node]

detect_category(rewritten_query) → category_filter

hybrid_retrieve(query, top_k=6, category_filter=cat)

Output: top-6 relevant document chunks

|

▼

[rag_generate_node]

Context = format_context(docs) ← max 6,000 characters

LLM (temp=0.05), STRICT grounding rules:

[PASS] Only use information from the provided documents

[N0] Never invent figures, rates, or product names not in documents

→ If not found: "I couldn't find that information. Please contact
our hotline or visit a branch."

→ END

Grounding enforcement: The system prompt explicitly forbids the model from answering based on general training knowledge. If documents don't contain the answer, the model must use the not-found template — not guess.

9.4 PRODUCT_CONSULT — Advisory Pipeline (Multi-Turn)

Phase 1: Domain Detection & Profile Recall

[advisor_domain_detector]

```

LLM (temp=0.0): classify into credit_card | insurance | loan | savings | general
|
▼
[advisor_profile_recall] ← V3 feature
  Load profile from AdvisorProfileStore for (user_id, domain)
  |
  └─ Profile EXISTS:
    |   Populate collected_info from stored profile
    |   Set awaiting_profile_confirm = True
    |   Send confirm message → END (wait for user response)
    |
    └─ No profile:
      Continue to field_extractor

```

Profile confirmation message format:

"I see that you previously consulted about [Credit Cards] and provided:

- Monthly income: [value]
- Primary spending: [value]
- Preferred benefits: [value]
- Existing card: [value]

Is this information still accurate?

- If yes, reply 'OK' and I'll advise you right away!
- If anything changed, just let me know."

Phase 2: Field Extraction & Collection

```

[advisor_field_extractor]
  required_fields = KnowledgeGraph.get_fields(domain)
  Pre-extract from initial question using LLM (temp=0.0):
    "I want a credit card, I earn 15M/month and prefer cashback"
    → collected_info = { "monthly_income": "15M", "preferred_benefits": "cashback" }
    → missing_fields = [ "primary_spending", "existing_card" ]
  Set turn_count = 0
  → advisor_collect_info
  |
  ▼
[advisor_collect_info] ← MULTI-TURN LOOP
  turn_count = 0:

```

```

missing_fields empty → "Finding options for you now!" → advisor_retrieve
missing_fields not empty → ask first missing field (with intro) → END
turn_count > 0:
    Save last user message → collected_info[missing_fields[0]]
    Update missing_fields (remove answered field)
    Still missing → ask next field → END
    All collected or max_turns reached → advisor_retrieve

```

Required fields per domain:

Domain	Required Fields
credit_card	Monthly income, Primary spending category, Preferred benefits (cashback/points/installment), Existing credit card
insurance	Age, Family status (married/children), Insurance purpose (health/savings/life/accident), Monthly budget
loan	Loan purpose (home/car/personal/business), Amount needed, Monthly income, Collateral available
savings	Amount to deposit, Time horizon (1/3/6/12 months/long-term), Savings goal (safety/high yield/flexible)
general	Product type of interest

Phase 3: Retrieval & Recommendation

[advisor_retrieve_node]

```

query = "{domain_keyword} {collected_info values joined}"
hybrid_retrieve(top_k=4, domain_hint=domain_kw, category_filter=domain)
Output: top-4 relevant product docs
|
▼

```

[advisor_recommend_node]

Check document relevance: if no relevant docs → not-found message + contact info

```

Build customer profile string from collected_info
LLM (temp=0.1), STRICT grounding:
  → Recommend specific named products from documents
  → Explain why each product fits the customer's profile
  → Never invent rates, benefits, or product names
Save profile: AdvisorProfileStore.save(user_id, domain, collected_info)
→ END

```

9.5 Profile Update Flow

When `awaiting_profile_confirm = True`, the START router sends the user's response to `advisor_profile_update_node`:

```

[advisor_profile_update_node]
  LLM (temp=0.0) extracts from user response:
  {
    "confirmed": true/false,
    "updates": { "field_name": "new_value", ... }
  }
  |
  |— confirmed = False:
  |   Reset: collected_info = {}, missing_fields = all_fields
  |   awaiting_profile_confirm = False, turn_count = 0
  |   → advisor_collect_info (start fresh)
  |
  |— confirmed = True, updates = {}:
  |   "Information confirmed, finding options now!"
  |   awaiting_profile_confirm = False
  |   → advisor_retrieve
  |
  |— confirmed = True, updates = {...}:
  |   Merge updates into collected_info
  |   AdvisorProfileStore.update_profile(user_id, domain, updates)
  |   "Updated [fields]. Finding options now!"
  |   awaiting_profile_confirm = False
  |   Check missing_fields → advisor_collect_info or advisor_retrieve

```


9.6 CUSTOMER_FEEDBACK Pipeline

This pipeline is called for customer feedback about the chatbot or bank products. It **bypasses cache entirely** — feedback responses must be contextual and personalized.

(skip cache)

|

▼

[customer_feedback_node]

Input: current message + last 4 conversation turns

LLM (temp=0.3) analyzes:

sentiment ∈ { NEGATIVE, POSITIVE, NEUTRAL }

subject: chatbot behavior OR bank product/service

|

└─ NEGATIVE (frustration, complaint, dissatisfaction):

|

→ Sincere apology, empathy

|

→ Acknowledge the specific issue raised

|

→ Commit to improvement

|

→ For product issues: offer hotline/branch

|

→ Tone: humble, gentle

|

└─ POSITIVE (praise, satisfaction, compliment):

|

→ Genuine thanks, humility

|

→ Ask if customer needs anything else

|

→ Tone: warm, not overly effusive

|

└─ NEUTRAL (ambiguous, no clear emotion):

→ Acknowledge the feedback

→ Ask for more detail

→ END

Examples:

Customer Message	Classified As	Response Style
"Why did you ask my loan purpose again? I already said home purchase."	NEGATIVE	Sincere apology + acknowledge the specific error

Customer Message	Classified As	Response Style
"This chatbot is really easy to use and answers clearly!"	POSITIVE	Genuine thanks + ask if they need more help
"I just tried this service for the first time."	NEUTRAL	Acknowledge + invite questions
"Your loan rates are higher than other banks."	NEGATIVE (product)	Empathize + offer to connect with specialist

10. Advisor Profile Store

10.1 Storage Format

Profiles are stored in JSON files, one file per user:

data/conversations/{user_id}_profiles.json

```
{
  "user_id": "UID001",
  "profiles": {
    "credit_card": {
      "monthly_income": "15 million",
      "primary_spending": "online shopping, dining",
      "preferred_benefits": "cashback",
      "existing_card": "none",
      "_updated_at": "2026-06-07T14:00:00"
    },
    "loan": {
      "loan_purpose": "home purchase",
      "amount_needed": "2 billion",
      ...
    }
  }
}
```

10.2 API Methods

Method	Signature	Called By
save_profile	(user_id, domain, collected_info)	advisor_recommend_node — after recommendation completes
load_profile	(user_id, domain) → dict \ None	advisor_profile_recall_node — before field extraction
update_profile	(user_id, domain, updates)	advisor_profile_update_node — when user updates specific fields

10.3 Profile Lifecycle

1. First-time credit_card consultation:
field_extractor → collect_info loop → retrieve → recommend
→ On completion: save_profile(uid, "credit_card", collected_info)
2. Second visit asking about credit cards:
profile_recall: load_profile(uid, "credit_card") → found
→ Show confirmation message → END (wait for user response)
3. User responds "OK" or with updates:
profile_update: merge any updates → advisor_retrieve (no re-questioning)
4. User wants to start fresh ("No, please ask me again"):
profile_update: confirmed=False → field_extractor → collect_info from scratch

10.4 Future Extension: Profile TTL

Currently profiles have no expiry. To add time-based expiration:

```
# advisor_profile_store.py – load_profile()
from datetime import datetime, timedelta

updated_at = datetime.fromisoformat(profile["_updated_at"])
if datetime.now() - updated_at > timedelta(days=90):
    return None    # profile too old, start fresh questioning
```

11. LangGraph State & Topology

11.1 ChatState TypedDict

```
class ChatState(TypedDict):

    # — Core —————
    messages          : Annotated[list, add_messages]
    session_id        : str
    user_id           : Optional[str]          # V3: for profile load/save per user

    # — Intent —————
    intent            : Optional[str]          # 5 possible values

    # — RAG —————
    rewritten_query    : Optional[str]
    retrieved_docs     : Optional[list]        # [{content, score_rank, category}]

    # — Advisor —————
    advisor_domain     : Optional[str]          # credit_card | insurance | loan | s
    required_fields    : Optional[dict]        # {field_name: question_to_ask}
    collected_info     : Optional[dict]        # {field_name: user_answer}
    missing_fields     : Optional[list]
    awaiting_profile_confirm : Optional[bool]    # V3: True while waiting for profile

    # — Cache —————
    cache_hit          : Optional[bool]
    cache_similarity    : Optional[float]

    # — Session —————
    turn_count         : int
    max_turns          : int                  # default 8
```

Key state behaviors:

- `messages` uses LangGraph's `add_messages` reducer — new messages are appended, not replaced
- `session_id` is used as `thread_id` for the MemorySaver checkpointer — each ses-

sion has isolated state

- `awaiting_profile_confirm = True` sets the highest-priority route at the START node
- `required_fields != None AND missing_fields != []` sets the second-priority route (continue active advisory)

11.2 Graph Topology — Edge Table

From	Condition	To
START	<code>awaiting_profile_confirm = True</code>	<code>advisor_profile</code>
START	<code>required_fields ≠ null AND missing_fields ≠ []</code>	<code>advisor_collect</code>
START	default	<code>intent_classifier</code>
<code>intent_classifier</code>	PRODUCT_CONSULT	<code>advisor_domain</code>
<code>intent_classifier</code>	CUSTOMER_FEEDBACK	<code>customer_feedb</code>
<code>intent_classifier</code>	others	<code>cache_check</code>
<code>cache_check</code>	<code>cache_hit = True</code>	END
<code>cache_check</code>	GREETING_FAREWELL miss	<code>greeting</code>
<code>cache_check</code>	PERSONAL_UNRELATED miss	<code>personal_unrel</code>
<code>cache_check</code>	PRODUCT_INFO_QA miss	<code>rag_rewrite</code>
<code>rag_rewrite</code>	always	<code>rag_retrieve</code>
<code>rag_retrieve</code>	always	<code>rag_generate</code>
<code>rag_generate</code>	always	END
<code>greeting</code>	always	END
<code>personal_unrelated</code>	always	END
<code>customer_feedback</code>	always	END
<code>advisor_domain_detector</code>	always	<code>advisor_profile</code>
<code>advisor_profile_recall</code>	profile found	END
<code>advisor_profile_recall</code>	no profile	<code>advisor_field</code>
<code>advisor_field_extractor</code>	always	<code>advisor_collect</code>
<code>advisor_collect_info</code>	still missing fields	END
<code>advisor_collect_info</code>	all collected	<code>advisor_retrie</code>
<code>advisor_retrieve</code>	always	<code>advisor_recomm</code>
<code>advisor_recommend</code>	always	END
<code>advisor_profile_update</code>	<code>confirmed=True, complete</code>	<code>advisor_retrie</code>
<code>advisor_profile_update</code>	<code>confirmed=True, missing</code>	<code>advisor_collect</code>
<code>advisor_profile_update</code>	<code>confirmed=False</code>	<code>advisor_collect</code>

11.3 MemorySaver & Persistence

MemorySaver (in-memory checkpointer) persists the graph state per `session_id`. This enables:

- Multi-turn advisory without re-sending full conversation history
- Resuming a mid-conversation advisory after the user sends a follow-up
- The `awaiting_profile_confirm` flag surviving across HTTP requests

Limitation: Server restart clears all in-memory state. Ongoing advisory sessions are lost. JSON history and profiles are unaffected.

Upgrade path for production:

```
# main_graph.py
from langgraph.checkpoint.sqlite import SqliteSaver
memory = SqliteSaver.from_conn_string("./data/checkpoints.db")
graph = builder.compile(checkpointer=memory)
```

12. API Layer

12.1 Endpoints

Method	Path	Description
GET	/	Basic health check
GET	/admin/health	Vectorstore status, model info, category counts
GET	/admin/stats	Chunk count, category breakdown, history statistics
POST	/admin/load	Load/reload parquet → ChromaDB
POST	/admin/load?force_reload= True	Force reload (required after updating parquet)
GET	/admin/history/{user_id}	Full Q&A history for a user (admin)
POST	/admin/rebuild-history	Clear ChromaDB history and rebuild from JSON files
POST	/chat/	Main chat endpoint
GET	/chat/session/{session_id}	Current graph state for a session
GET	/chat/history/{user_id}	Chat history for a user
POST	/chat/new	Create a new session

12.2 Chat Endpoint — Request / Response

Request:

```
{
  "message"      : "I want to open a credit card but don't know which one",
  "session_id"   : "abc-123",
  "user_id"      : "UID001"
}
```

Response:


```
{
  "session_id"      : "abc-123",
  "answer"          : "To recommend the right card for you, I need a few details...",
  "intent"          : "PRODUCT_CONSULT",
  "advisor_domain"  : "credit_card",
  "sources"         : [],
  "turn_count"      : 1,
  "collected_info" : {},
  "from_cache"      : false,
  "cache_similarity" : null
}
```

12.3 Post-Graph Processing

After `graph.invoke()` returns, the API route performs:

```
result = graph.invoke(
    {
        "messages" : [HumanMessage(content=message)],
        "session_id" : session_id,
        "user_id" : user_id
    },
    config={"configurable": {"thread_id": session_id}},
)

# Determine if we're still in a multi-turn collection phase
missing_fields = result.get("missing_fields") or []
is_collecting = bool(missing_fields)

# Save Q&A to history only if:
# 1. We have a final answer (not mid-advisory)
# 2. Answer was not served from cache (avoid duplicates)
if not is_collecting and not cache_hit and answer:
    store.save(user_id, question, answer, intent, advisor_domain, session_id)
```

13. Evaluation Framework

13.1 Overview

The evaluation framework covers all pipeline layers independently. A failure at one layer invalidates results from subsequent layers.

Evaluation order: 1. Intent classification (Step 1a) — must pass before Steps 2-4 are meaningful 2. Cache (Step 1b) — independent of intent accuracy 3. Customer feedback sentiment (Step 1c) — independent 4. RAG pipeline (Step 2) — depends on intent routing working correctly 5. Advisory field collection (Step 3) — depends on intent + domain detection 6. Recommendation quality (Step 4) — uses output of Step 3 as input

General evaluation principles:

- Use dedicated eval user IDs (EVAL_ prefix) to avoid polluting production history
- All eval scripts call the live API — the server must be running
- Ollama must be running — some eval steps use LLM-as-judge
- Evaluate against the smallest failing layer first

13.2 Evaluation Scorecard

Step	Component	Primary Metric	Target
1a	Intent Classification	Macro F1	≥ 0.85
1b	Cache	False Positive Rate	≤ 0.05
1c	Customer Feedback Sentiment	Accuracy	≥ 0.85
2	RAG (Retrieval + Generation)	Faithfulness	≥ 0.80
3	Advisory Field Collection	Completion Rate	≥ 0.90
4	Advisory Recommendation	Correct Rate	≥ 0.70

13.3 Step-by-Step Metrics

Step 1a — Intent Classification

Metrics collected: - Overall accuracy (correct/total) - Per-intent Precision, Recall, F1 - Confusion matrix (identify which intent pairs are frequently confused)

Common confusion pairs: - PERSONAL_UNRELATED ↔ PRODUCT_CONSULT — customer mentions financial situation; does it include a product request? - PRODUCT_INFO_QA

↔ PRODUCT_CONSULT — does the customer want a fact or a recommendation? - CUSTOMER_FEEDBACK ↔ PERSONAL_UNRELATED — subjective statement vs. sharing context

Red flag: $F1 < 0.70$ for any single intent → review prompt examples for that intent in `intent_classifier.py`

Step 1b — Cache Evaluation

Two test pair types:

Paraphrase pairs (should hit cache — `should_hit: true`):

Stored: "What is the annual fee for the Cashback Card?"

New: "How much is the yearly fee for the Cashback Card?"

→ Expected: HIT (same question, rephrased)

Near-miss pairs (should NOT hit cache — `should_hit: false`):

Stored: "What is the annual fee for the Cashback Card?"

New: "What is the credit limit for the Cashback Card?"

→ Expected: MISS (same card, different information)

Threshold sensitivity test: Run evaluation at thresholds 0.75 / 0.80 / 0.85 to find the optimal trade-off between TPR and FPR.

Red flag: $FPR > 0.10$ → increase `CACHE_SIMILARITY_THRESHOLD` or re-examine the hybrid scoring weights.

Step 2 — RAG Quality (LLM-as-Judge)

Four metrics scored by an LLM judge:

Metric	Definition	Scale
Answer Relevance	Does the answer address what was asked?	1-5
Faithfulness	Does the answer stay within the retrieved documents? (no hallucination)	0 / 1

Metric	Definition	Scale
Completeness	Does the answer cover all necessary information?	1-5
Groundedness	Are all claims traceable to a source document?	1-5

Faithfulness is binary and critical for a banking context — a single hallucinated interest rate is unacceptable.

RAGAS integration: The eval script supports `--use-ragas` flag for the full RAGAS framework (requires `pip install ragas`).

Step 3 — Advisory Field Collection

Metrics:

- **Field Completion Rate** = fields successfully collected / fields required
 - Target: ≥ 0.90 (allows 1 missed field per ~10 scenarios)
- **Turn Efficiency** = actual turns taken / fields required
 - Ideal: ~1.5 turns per field (some context required)
 - Red flag: >3 turns per field → bot is inefficient or repetitive
- **Re-ask Rate** = turns spent asking for already-provided info / total turns
 - Target: ≤ 0.10 — bot should not re-ask what it already knows
 - Red flag: >0.20 → check `field_extractor.py` pre-extraction logic

Simulation approach: The evaluation script plays the role of the customer, automatically providing answers from a predefined `user_profile` in `advisory_scenarios.json`. The bot doesn't know these answers in advance — it must ask for them.

Sample advisory scenario:

```
{
  "id": "adv_001",
  "domain": "credit_card",
  "opening_message": "I want to get a credit card but am not sure which one to choose",
  "user_profile": {
    "monthly_income": "15 million",
```

```
"primary_spending": "online shopping and dining",
"preferred_benefits": "cashback",
"existing_card": "none"
}
}
```

Step 4 — Recommendation Quality (LLM-as-Judge)

Since there is no single “correct” product recommendation, quality is evaluated by an LLM judge using:

Judge evaluation criteria:

1. **Relevance** — Does the recommended product match the customer’s stated needs and profile? (Correct / Partial / Incorrect)
2. **Groundedness** — Is the recommendation supported by the retrieved document excerpts? (Grounded / Hallucinated)
3. **Completeness** — Were any obviously better products overlooked? (No / Yes)

Judge receives: - Customer’s collected_info (full profile) - The recommendation text - The retrieved document context that was available

Target: Correct Rate ≥ 0.70 , Hallucination Rate ≤ 0.15

13.4 Running the Evaluation

Prerequisites

```
# Start services (Windows)
ollama serve                    # Terminal 1
cd version_3 && start_api.bat    # Terminal 2

# Verify health
curl http://localhost:8000/admin/health
# → { "status": "ok", "vectorstore_ready": true, ... }

# Install eval dependencies
pip install requests scikit-learn tabulate colorama
```

Individual Steps

```
cd version_3

# Step 1a – Intent
python evaluation/step1_intent_eval.py

# Step 1b – Cache
python evaluation/step1_cache_eval.py --seed-first

# Step 1c – Feedback sentiment
python evaluation/step1_feedback_eval.py --with-quality-check

# Step 2 – RAG quality
python evaluation/step2_rag_eval.py

# Step 3 – Advisory field collection
python evaluation/step3_advisory_eval.py

# Step 4 – Recommendation quality (uses Step 3 output)
python evaluation/step4_recommend_eval.py \
    --sessions evaluation/results/step3_advisory.json
```

Full Pipeline Run

```
python evaluation/run_all.py \
    --steps 1a 1b 1c 2 3 4 \
    --output-dir evaluation/results/run_$(date +%Y%m%d_%H%M%S) \
    --fail-fast
```

Sample full run output:

```

┌────────────────────────────────────────────────────────────────────────────────┐
│ Bank Chatbot V3 – Evaluation Report                                           │
│ Run: 2026-06-10 14:30:22                                                       │
└────────────────────────────────────────────────────────────────────────────────┘
┌ Step 1a Intent Classification Macro F1 0.87 [PASS] ───────────────────────────┐
┌ Step 1b Cache FPR 0.07 [WARN] ─────────────────────────────────────────────────┐

```

Step 1c	Feedback Sentiment	Accuracy	0.89	[PASS]	
Step 2	RAG Faithfulness	Score	0.82	[PASS]	
Step 3	Advisory Completion	Rate	0.94	[PASS]	
Step 4	Recommendation Quality	Correct	0.75	[PASS]	
=====					
Overall: 5 PASS 1 WARN					
=====					

13.5 Re-running After Changes

Change	Steps to Re-run
Modified intent_classifier.py prompt	1a
Changed CACHE_SIMILARITY_THRESHOLD	1b
Modified customer_feedback.py prompt	1c
Updated product documents (parquet)	2
Changed field_definitions.py	3
Modified recommender.py prompt	4
Deploying to a new environment	1a + 1b + 2 (minimum)
Full release regression	All steps (run_all.py)

14. Configuration Reference

14.1 Key Settings

.env Key	Default	Description
LLM_MODEL	deepseek-r1:8b	Main LLM model via Ollama
OLLAMA_NUM_GPU	-1	GPU layers: -1 = all GPU, 0 = CPU only
LLM_NUM_CTX	4096	LLM context window (tokens)
EMBEDDING_MODEL	bge-m3:latest	Embedding model (1024-dim)
CHROMA_COLLECTION_NAME	products_v3	Product knowledge ChromaDB collection
TOP_K_RETRIEVAL	6	Documents fetched for RAG
TOP_K_FINAL	4	Documents fetched for advisory
SEMANTIC_WEIGHT	0.65	Semantic weight in RRF fusion
BM25_WEIGHT	0.35	BM25 keyword weight in RRF fusion
CACHE_SIMILARITY_THRESHOLD	0.8	Minimum hybrid score for cache hit
CACHE_TOP_K	5	Candidates retrieved for cache lookup
MAX_ADVISOR_TURNS	8	Maximum turns in advisory collection loop
API_RELOAD	true	uvicorn hot-reload (disable in production)

14.2 Performance Trade-offs

Setting	Increase →	Decrease →
LLM_NUM_CTX	Better reasoning, longer context	Significantly slower (DeepSeek-R1 thinks in full context)
TOP_K_RETRIEVAL	More candidate documents, better recall	Slower, larger prompts
SEMANTIC_WEIGHT	Better for semantically paraphrased queries	Less keyword precision
CACHE_SIMILARITY_THRESHOLD	Fewer false cache hits	More cache misses (more LLM calls)
MAX_ADVISOR_TURNS	Bot can ask more fields	Longer conversations, slower resolution

Critical note on LLM_NUM_CTX: DeepSeek-R1 generates <think> blocks that scale with context size. At num_ctx=8192, advisory scenarios that took ~112s at num_ctx=4096

took over 600s (evaluation timeout). Default is set to 4096 for all nodes.

15. Project Lessons Learned

15.1 Challenges Encountered

DeepSeek-R1 Think Tokens

Problem: DeepSeek-R1 outputs `<think>...</think>` blocks before its final answer. If this output is passed as input to another LLM call (e.g., as the answer text in a judge prompt), Ollama's llama-server rejects it with HTTP 500.

Solution: Strip think tokens at every node that produces LLM output:

```
import re
answer = re.sub(r"<think>.*?</think>", "", response.content, flags=re.DOTALL).strip()
```

Applies to: `rag_generate_node`, `advisor_recommend_node`, and any node whose output may be reused as input.

Context Window and Latency

Problem: Setting `num_ctx=8192` in `field_extractor.py` increased per-scenario evaluation time from ~112s to >600s (evaluation timeout). DeepSeek-R1's thinking token generation scales with context window size.

Solution: Use the default `num_ctx=4096` (4096 is sufficient for field extraction prompts which are short). Only increase context window when processing very long documents.

Edit Tool File Truncation

Problem: The file editing tool truncated files at certain Unicode characters (em dashes, Vietnamese accented characters in strings), leaving files syntactically broken with null bytes.

Solution: - Avoid Unicode characters in Python string literals; use ASCII equivalents in source code - Use bash heredoc (`cat > file.py << 'PYEOF'`) for complete file rewrites - Run `ast.parse()` + null byte check after every file edit as a validation step

LangGraph sync/async mismatch

Problem: LangGraph's `graph.invoke()` is synchronous and blocks the FastAPI async event loop when called from an `async def` route handler.

Solution: Wrap the invoke call in `asyncio.get_event_loop().run_in_executor(None, ...)` or use FastAPI's `BackgroundTasks / run_in_executor` pattern. Alternatively, run the FastAPI server with `--workers 1` and accept the blocking behavior for a single-instance deployment.

Cache False Positive Rate

Problem: Pure semantic similarity produced FPR of ~0.12 (12% of distinct questions incorrectly hitting cache).

Solution: Hybrid scoring (0.65 semantic + 0.35 Jaccard keyword) reduced FPR to 0.07. Still above the 0.05 target — consider further increasing the threshold or weighting.

15.2 Design Decisions

Why in-graph cache instead of API-level cache

Moving cache to a LangGraph node allows:

- Access to the full graph state (intent, session, user context)
- Selective bypass: `PRODUCT_CONSULT` and `CUSTOMER_FEEDBACK` skip the cache node entirely via routing logic
- Unified logging — cache hits appear in the same log stream as all other nodes
- Easier testing — the cache node can be unit-tested with a mock state

Why JSON files instead of a database for profiles/history

- **Zero infrastructure dependency** — no PostgreSQL, Redis, or other service required
- **Human-readable** — developers can inspect and manually correct stored data
- **Simple backup** — files can be copied; no database dump required
- **Sufficient for scale** — at 1,000 users × 100 entries each, total JSON is <50MB

For production scale (>100,000 users), migrating to PostgreSQL or MongoDB would be straightforward with the existing `ConversationStore / AdvisorProfileStore` interface.

Why category-level filtering instead of re-ranking

Re-ranking (cross-encoder scoring) would be more accurate but adds 200-500ms per query. For a local LLM stack already spending 20-30s per LLM call, retrieval latency is not the bottleneck. Category filtering provides a good accuracy improvement at near-zero additional cost.

15.3 Future Improvements

Short-term

- **Cache threshold tuning** — systematic A/B testing to find optimal threshold per intent type (currently one threshold for all)
- **Field extraction quality** — evaluate pre-extraction accuracy per domain; add more few-shot examples for domains with low extraction precision
- **Streaming responses** — FastAPI Server-Sent Events (SSE) for token streaming, reducing perceived latency for slow LLM calls

Medium-term

- **SqliteSaver checkpointer** — replace MemorySaver with SqliteSaver for advisor session persistence across server restarts
- **Profile TTL** — add 90-day expiry to advisor profiles; stale profiles re-trigger the collection flow
- **Additional advisory domains** — extend `field_definitions.py` for new product categories (investment funds, foreign exchange, etc.)
- **Feedback loop** — surface negative-sentiment feedback cases for human review; use confirmed errors to improve prompts

Long-term

- **ChromaDB HTTP server mode** — separate ChromaDB into its own service for horizontal scalability
- **Redis-backed checkpointer** — for multi-instance FastAPI deployment
- **A/B testing framework** — serve different prompt variants to different user segments, measure intent accuracy and advisory completion rate
- **RAG with re-ranking** — add a cross-encoder reranker for higher retrieval precision when the document corpus grows significantly

16. Project Structure

```

version_3/
├─ config/
│   └─ settings.py                # pydantic-settings, all config keys
├─ scripts/
│   └─ check_gpu.py              # GPU utilization check utility
├─ src/
│   ├─ llm.py                    # get_llm(), get_fast_llm(), parse_json()
│   ├─ data/
│   │   └─ loader.py             # Parquet → ChromaDB + BM25 (category metadata)
│   │   └─ retrieval/
│   │       └─ retriever.py       # hybrid_retrieve() with RRF fusion
│   │   └─ knowledge_graph/
│   │       └─ field_definitions.py # DOMAIN_FIELDS, DOMAIN_KEYWORDS, DOMAIN_LABELS
│   │   └─ history/
│   │       └─ conversation_store.py # JSON + ChromaDB history, cache lookup, sync
│   │       └─ advisor_profile_store.py # Advisor profile persistence per user+domain
│   │   └─ graph/
│   │       └─ state.py           # ChatState TypedDict
│   │       └─ main_graph.py      # LangGraph assembly, all routing functions
│   │       └─ nodes/
│   │           └─ intent_classifier.py # 5-intent classification with LLM
│   │           └─ greeting.py       # GREETING_FAREWELL with farewell keyword check
│   │           └─ personal_unrelated.py # Empathy + bridge to product
│   │           └─ cache_check.py     # In-graph cache node (hybrid similarity)
│   │           └─ customer_feedback.py # Sentiment analysis + contextual response
│   │           └─ rag/
│   │               └─ rewrite.py     # Query rewriting with conversation context
│   │               └─ retrieve.py    # Category-filtered hybrid retrieval
│   │               └─ generate.py    # Strict-grounded answer generation
│   │               └─ advisor/
│   │                   └─ domain_detector.py # Domain classification: 5 categories
│   │                   └─ profile_recall.py # Load + confirm stored profile
│   │                   └─ field_extractor.py # Required fields + LLM pre-extraction
│   │                   └─ info_collector.py # Multi-turn field collection loop
│   │               └─ recommender.py # Category-filtered retrieval + recommendation

```

```

├── api/
│   ├── main.py                # FastAPI app, startup events
│   ├── models/schemas.py     # Pydantic request/response models
│   └── routes/
│       ├── chat.py            # POST /chat/, history, session endpoints
│       └── admin.py           # /admin/load, /admin/health, rebuild-history
├── frontend/
│   └── app.py                  # Streamlit chat UI
├── evaluation/
│   ├── data/
│   │   ├── intent_samples.json # ~60 labeled intent samples
│   │   ├── rag_samples.json    # ~20 Q&A with ground truth answers
│   │   ├── cache_pairs.json    # Paraphrase + near-miss test pairs
│   │   ├── feedback_samples.json # ~30 labeled feedback samples
│   │   └── advisory_scenarios.json # 5 multi-turn advisory simulations
│   ├── results/                # Eval output (gitignored)
│   ├── utils.py                # API client, LLM judge, reporter
│   ├── step1_intent_eval.py
│   ├── step1_cache_eval.py
│   ├── step1_feedback_eval.py
│   ├── step2_rag_eval.py
│   ├── step3_advisory_eval.py
│   ├── step4_recommend_eval.py
│   └── run_all.py              # Orchestrates all steps, prints scorecard
├── data/
│   ├── documents_bgem3.parquet # Pre-embedded product knowledge base
│   ├── vectorstore/           # ChromaDB persistent storage
│   └── conversations/
│       ├── {user_id}.json      # Q&A history per user
│       └── {user_id}_profiles.json # Advisory profile per user
├── requirements.txt
├── .env / .env.example
├── start_api.bat
└── start_streamlit.bat

```

Appendix A: parse_json — DeepSeek-R1 Output Handling

DeepSeek-R1 generates `<think>...</think>` reasoning blocks before its final answer. The `parse_json()` utility handles this across all nodes that expect JSON output:

1. Strip all `<think>...</think>` blocks from the response
2. Attempt `json.loads()` on the remaining text
3. If that fails: find all `{...}` blocks → try from the LAST one upward
(DeepSeek puts the actual answer after the thinking, so the last block is most likely to be the valid JSON)
4. If that fails: look for ````json ... ```` code fence
5. If all fail: return `{}` and log a warning

This strategy is used by: - `intent_classifier.py` — parsing `{"intent": "PRODUCT_CONSULT"}` - `field_extractor.py` — parsing `{"field_name": "value", ...}` - `advisor_profile_update.py` — parsing `{"confirmed": true, "updates": {...}}` - `customer_feedback.py` — parsing `{"sentiment": "NEGATIVE", "response": "..."}

---`

Appendix B: Extending the System

Adding a New Advisory Domain

1. Add keywords to `CATEGORY_KEYWORDS` in `src/data/loader.py`
2. Add `DOMAIN_FIELDS[new_domain]` in `field_definitions.py` — define the required fields and their question prompts
3. Add `DOMAIN_KEYWORDS[new_domain]` and `DOMAIN_LABELS[new_domain]`
4. Add the new domain option to the LLM prompt in `domain_detector.py`
5. Run `POST /admin/load?force_reload=true` to re-index documents with the new category

Adding a New Intent

1. Add the intent name to `VALID_INTENTS` in `intent_classifier.py`
2. Add description and examples to `CLASSIFY_PROMPT`
3. Create a new node file in `src/graph/nodes/`
4. Add the node to `main_graph.py` and add routing edges
5. Add evaluation samples to `evaluation/data/intent_samples.json`
6. Re-run Step 1a evaluation

Upgrading the LLM

In .env:

```
LLM_MODEL=deepseek-r1:14b    # More accurate, ~2x slower  
LLM_MODEL=llama3.2:3b        # ~3x faster, lower accuracy  
LLM_MODEL=qwen2.5:7b         # Good Vietnamese, fast
```

After changing model:

1. Run ollama pull <new_model>

2. Restart API server

3. Run full evaluation: python evaluation/run_all.py